[Experiment, Analysis, and Benchmark] GeoScopeJoin: When Does Exact Spatial Query Reuse Pay?

Hangyu Du, Si Qiao*
The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China
dominohydu@hkust-gz.edu.cn, siqiao@hkust-gz.edu.cn
ORCID: 0009-0005-5499-3348 (H. Du); 0000-0002-2002-3678 (S. Qiao)
*Corresponding author

***Abstract*—Spatial queries often join point data with raster datasets that are revised independently. Recomputing every result after each revision is costly, while reuse can produce incorrect rows or outdated provenance when points or raster cells change. This setting calls for a joint account of query correctness, source identity and end-to-end service cost. We address these questions through an executable correctness contract for output rows, an exact identity for evaluation counts, and lifecycle measurements that include data preparation and query processing. In controlled experiments, the counting identity holds exactly in 90 runs, and deferred maintenance reduces service time by 16.4% at the tested endpoint with high overlap and delayed queries. Across 720 real-data sessions, no configuration meets the protocol-specified 5% end-to-end improvement threshold, as preparation accounts for 93–98% of one-shot lifecycle cost. Reusing registered snapshots across 100 requests reduces per-request cost by 12–31× through amortization of preparation ($n = 7$ sessions per cell; $n = 30$ for $H = 1$). These results identify when exact reuse saves work and when those savings translate into faster service.**

***Index Terms*—spatial queries, incremental maintenance, data revisions, provenance, experimental validation**

I. Introduction

Spatial analyses often combine a raster release with a separately maintained collection of point objects. Repeating an analysis after either source changes invites reuse: much of the spatial relationship may remain unchanged. A reusable result must preserve both its numerical value and its source evidence. A point may move across a cell boundary, a newly inserted point may enter a previously empty region, or a new release may preserve the value while changing the evidence that must accompany it. Cross-version reuse is therefore a joint problem of query semantics, dependency maintenance, source identity, and the cost of obtaining a trustworthy snapshot.

Consider a query that associates each point covered by a region with the population value of its containing raster cell. If two points occupy the same cell, the query returns two object-specific rows. If a point lies in two regions, it contributes a row to each. If a region covers a point outside the raster extent, the absence of coverage remains an explicit result. The query contract preserves each object, distinguishes missing states from zero, and attaches the selected release identifier to every row.

Incremental view maintenance supplies the general alternative to full recomputation [1]. Raster–vector processing and distributed raster systems address important execution problems [2], [3], while provenance-based data skipping and its incremental maintenance investigate how dependencies support reuse [4], [5]. We bring these directions together through a row-level contract for independently versioned point and raster inputs. The contract connects exact spatial results to the registered source bytes and provides a common basis for comparing maintenance strategies.

This paper asks three questions and answers them separately.

RQ1 (contract). What row-level evidence is required for an exact answer when the raster and point versions are selected independently?

RQ2 (mechanism). Under what preconditions does a conservative affected set suffice, and how can the benefit of deferred maintenance in evaluation counts be predicted?

RQ3 (service). Under a complete lifecycle and a fixed memory budget, where does the benefit of maintenance actually go, and when is it large enough to matter?

We answer RQ1 with an executable row contract whose source fields are bound to the SHA-256 of the source archive and of the point file, and we test it with a source-only change that must alter the suffix while preserving the semantic prefix. To address RQ2, we establish a counting identity for fixed point sets, verify it to zero residual on a controlled overlap matrix, and validate its boundary with paired timing at three protocol-specified endpoints. Finally, the practical service implications (RQ3) are evaluated through 720 sessions analysed in paired blocks across two datasets, two service routes, two schedules and three engines, a tile cache operating under a frozen byte budget, and a real movement across a cell boundary sampled from the native raster.

The counting identity predicts the benefit in evaluation counts exactly, and the timing endpoints confirm the predicted service boundary: no benefit when the update overlap is zero or when every state is queried, and a stable 16% reduction when both overlap and deferral are present. On the real datasets, no one-shot configuration passes the protocol-

specified 5% improvement threshold: preparation, including source verification, decoding and encoding, consumes 93–98% of the lifecycle, so maintenance and querying together account for 0.7–4.2% of it. Reusing a registered snapshot reduces per-request cost by 12–31× over 100 requests by amortizing preparation; the controlled paired timings separately establish when deferral itself reduces service time. A tile cache that respects a hard byte budget is 18–26× slower per query than lazy maintenance at every budget, because its granularity is the tile rather than the changed object.

Together, these results identify the reusable benefit, the measurement needed to observe it, and the preparation costs that determine its end-to-end significance.

II. Related Work

a) Incremental maintenance: Maintaining views under updates is a longstanding database problem [1]. Our controls based on a reverse index and dirty sets instantiate dependency tracking and deferred execution for exact point–raster queries. The argument for affected sets specifies the preconditions for spatial correctness, and the counting identity predicts when deferral can reduce evaluation work.

b) Raster and vector processing: The Raptor join operator studies processing raster and vector data [2]; RDPro develops distributed processing for large raster workloads [3]. These works address spatial execution at the operator level and in distributed systems. Our focus is exact reuse for point samples within identified regions on compatible native grids.

c) Dependency summaries and provenance: PBDS captures provenance sketches for identifying query-relevant data and reusing that information [4]; IMP investigates incremental in-memory maintenance of such sketches [5]. Our source suffix complements this line of work by binding each row directly to selected input versions and hashes. The central distinction for reuse is between retaining semantic rows and refreshing their source evidence.

III. Problem Definition

The problem is to return exact point–raster query results for independently selected input versions while reusing valid semantic results.

A. Independently Versioned Inputs

Let R^r denote a raster release, P^p a point snapshot, G a fixed collection of identified regions, and S the spatial and numerical interpretation. The query is $Q(R^r, P^p, G, S)$. The raster and point versions are selected independently. The interpretation S includes the coordinate reference system, affine grid transform, cell assignment convention, region predicate, missing-data policy, and numerical encoding. Reuse across a change in S requires an additional compatibility argument; the maintenance experiments assume it is fixed.

Three notions of time remain separate. A raster observation epoch describes the phenomenon represented by the product. A release identifier distinguishes published revisions of that product. An OpenStreetMap (OSM) database timestamp identifies a recorded map state; it is not the real-world opening, closure, or movement time of a facility. Comparing two observation years cannot substitute for comparing two revisions of the same epoch.

For each region g and point identifier i whose projected location is covered by g, the output contains a row

$$(g, i, c, s, u, r, p, h_R, h_P), \tag{1}$$

where c is the cell identifier, s the status, u the encoded value, and h_R, h_P the registered source hashes. Cell identifiers are local to the extracted window. The emitted row binds the point snapshot through p and h_P.

B. Status, Geometry, and Multiplicity

The status is OBSERVED when a covering cell has a valid value, NODATA when the cell exists but carries the missing-value sentinel, and NOCOVERAGE when no cell in the loaded coverage contains the point. Both missing states have a null numerical field, but only NOCOVERAGE has no cell identifier. An observed zero remains OBSERVED with $u = 0$.

The implementation uses the `covers` region predicate. Polygon boundary points, including hole-boundary points, are covered; points strictly inside a hole are excluded. Multipart regions have union semantics within one region identifier. Overlapping regions generate distinct rows. Distinct point identifiers in one cell remain distinct. Canonical sorting facilitates comparison without deduplicating meaningful rows.

For the positive-axis internal grid representation, cell (j, k) covers $[x_0 + jd_x, x_0 + (j+1)d_x) \times [y_0 + kd_y, y_0 + (k+1)d_y)$. This representation converts the source raster's row orientation explicitly. East and north edges are excluded from a cell and assigned to the adjacent cell if present. Invalid geometries and unsupported grid interpretations are rejected or routed to a compatible full evaluation.

C. Per-Product Numerical Encoding

Valid values in the evaluated GHSL windows are represented as integers u with value $u/2^{32}$ when every value admits that exact representation. The encoder checks the exact ratio of the decoded floating-point value and rejects values whose binary denominators do not divide 2^{32}; its diagnostic identifies the denominator. Negative zero is rejected because the integer representation does not preserve its sign. The product's sentinel is mapped to NoData before encoding, and non-finite valid values are rejected.

One evaluated site (São Paulo) contains values that fail this test, for example 0.0008770349086262286. That site encodes the exact IEEE-754 bit pattern as sixteen hexadecimal digits and checks it against the array's bit view. The pipeline records the encoding selected for each product.

IV. Proposed Method

A. Overall Framework

Figure 1 organises the implemented workflow around the three questions in the introduction. The query contract and source binding address RQ1; change detection, affected sets

and eager or deferred maintenance address RQ2; and the two service routes expose the preparation and execution costs relevant to RQ3. Eager and Lazy provide alternative maintenance policies; an independent reference evaluates their outputs offline.

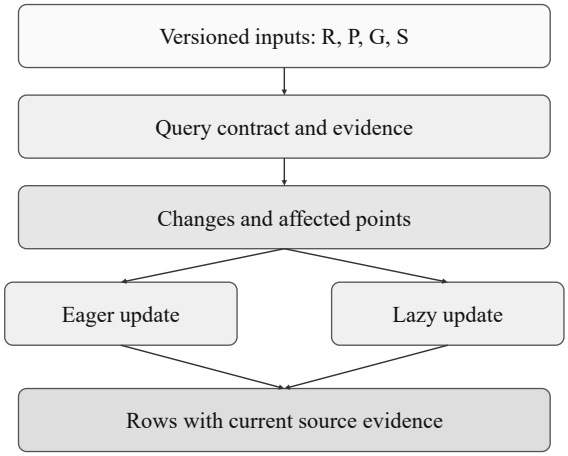


Fig. 1. GeoScopeJoin workflow. Inputs select independent raster and point versions and fixed region and interpretation rules. The contract defines valid rows and evidence; changes determine affected points; either Eager or Lazy maintains the semantic result. Output is bound to the requested source versions. Independent comparison with a reference is performed offline.

B. Query Semantics and Evidence Binding

We split each row into a semantic prefix (g, i, c, s, u) and a source suffix (r, p, h_R, h_P). The semantic prefix can be retained when its dependencies remain valid. The suffix must reflect the requested versions even if every semantic field is unchanged. A source-only revision updates the source suffix while retaining the valid semantic prefix.

A registered hash binds each result to the bytes recorded in the source registry. The archived Round 2 manifest `bench_freeze.json` records the window hashes and a self-reported freeze time of 2026-10-29 08:11:03 UTC; the raw route verifies decoded windows against them. The local manifest is included in the revision evidence package. This check establishes byte agreement with the registry; publisher authentication remains outside its scope.

C. Change Detection and Affected-Set Construction

Let Δ_R be a complete set of changed cell keys, including changes of value or status, and Δ_P a complete set of changed point identifiers, including insertions and deletions. Under fixed G and S, a conservative affected set contains every identifier in Δ_P and every surviving point mapped to a cell in Δ_R. An inverse map from cells to current point identifiers supports the latter lookup.

The sufficiency argument is direct. A surviving point outside that set has unchanged relevant attributes and an unchanged containing cell; with fixed geometry and interpretation its memberships, status and value are unchanged. A changed point must be reevaluated regardless of whether it appeared in an old result, and a deleted identifier must have its old rows removed. The argument requires complete deltas, a correct cell mapping and compatible snapshots.

Insertions show why positive dependencies alone are insufficient: a cached region result that is empty has no previous identifiers to follow, yet an insertion can make it nonempty. For movement, the implementation must remove the old entry in the inverse index, add the new one, invalidate old memberships and evaluate the new location. Invalidation by tile therefore considers both old and new tiles.

The optimization based on affected sets applies when the influence domain, deltas and grid semantics satisfy these preconditions. Otherwise, the affected set must be enlarged or the result rebuilt; global normalization, changing region definitions and arbitrary resampling require such a fallback.

D. Eager and Deferred Maintenance

Once the affected identifiers are known, Eager reevaluates them after each event, whereas Lazy records dirty identifiers and evaluates their union before producing a query result. This design targets repeated work when several events affect the same point before the next query. Both policies preserve the same row contract; the only intended difference is when semantic work is performed. The `B3-Lazy-demand-init` variant builds necessary dependencies at construction and defers the initial materialisation until the first query.

E. Evaluation-Count Analysis

For a fixed point set with no insertions or deletions, let n be the initial number of points, A_t the set of surviving points affected by event t, and B_j the group of events between two consecutive queries. The number of semantic evaluations is

$$C_E = n + \sum_t |A_t|, \qquad C_L = n + \sum_j \Big| \bigcup_{t \in B_j} A_t \Big| \tag{2}$$

for eager and deferred maintenance respectively. The identity is exact under the preconditions above and requires no timing. It yields three falsifiable predictions: with no shared core ($h = 0$) deferral cannot merge anything even at long query gaps; with a query after every event ($L = 1$) no merging across events is possible at any overlap; and the saving grows with both the shared core and the gap. Insertions, deletions, changes that alter only the source suffix, and complete fallbacks are not covered by the identity and are counted separately.

The identity also bounds what an experiment at the service level can show. If the maintenance work that deferral removes is a small fraction of the lifecycle, the end-to-end ratio stays near one even when the counter ratio is far from one. That is why both measurements are reported below.

F. Execution Workflow and Cost Model

A request selects the raster and point versions and verifies the relevant input evidence. The engine identifies affected points from changes and then applies either Eager or Lazy

maintenance. At query time it evaluates the necessary objects, combines retained semantic fields with the current version evidence, and emits the complete row set. If the compatibility or completeness conditions do not hold, the system must enlarge the affected set or fall back to full evaluation. The two routes below define the scope of the corresponding service measurements; snapshot reuse and result reuse remain distinct.

We distinguish two complete routes. The *raw route* starts from the source raster archives and point files: source verification, decoding, encoding, point preparation, delta discovery, engine build, updates, queries and a hash sink all lie inside the measured lifecycle. The *registered route* imports immutable snapshots whose integrity can be verified, pays a registration cost T_{register} once, and then re-verifies snapshot identity, the query contract and the version chain on every request. Its total service cost over H requests is

$$T_{\mathrm{reg,total}}(H) = T_{\mathrm{register}} + \sum_{j=1}^{H} T_{\mathrm{online},j}. \tag{3}$$

Reusing an input snapshot and reusing a cached query result are kept separate: the former amortizes preparation, while the latter changes what is recomputed within a request.

The lifecycle timer measures one outer wall-clock interval. Its top-level phases are mutually exclusive: source verification, decode, encode, point preparation, delta discovery, build, replay, and close. Update, query and serialization are nested within the replay phase, and lazy flushing is nested within query. The timer prohibits adding a parent interval to the sum of its children and stores the closure residual. When complete output has n rows, returning those rows requires $\Omega(n)$ work even if no semantic recomputation is necessary.

V. Experiments

We evaluate correctness and provenance (RQ1), the effect of overlap and deferred evaluation (RQ2), and the end-to-end service implications under the measured lifecycle (RQ3).

A. Datasets and Experimental Setup

1) Datasets and Workloads: **Shenzhen release pair.** The Shenzhen release pair consists of the GHSL population releases R2022A and R2023A for epoch E2020. Both releases use the 100 m Mollweide grid and tile R7_C29, and the evaluation uses a frozen 256×256-cell window at the origin $(10857900, 2744600)$. Three point snapshots (148, 175 and 183 nodes) give two events: a point-only event, followed by an event that changes both points and raster. The query uses four frozen regions (whole, hole, multipart, and an expanded envelope), which produce many NoCoverage rows. The envelope is the axis-aligned Mollweide min–max box of the four projected corners of the WGS84 movement-search box $[113.55, 114.35]^\circ$E $\times [22.2, 22.85]^\circ$N; the point snapshots were extracted from this search box. Shenzhen sessions on these snapshots are therefore postselected with respect to the movement search; the synthetic controlled matrix is separate from this event selection. R2022A has since been withdrawn upstream; the pair survives only because it was archived before withdrawal.

São Paulo sequence across epochs. The São Paulo sequence comprises the GHS-POP R2023A epochs E2015, E2020, E2025 and E2030 on tile R12_C14. The evaluation window is centred on the coordinate-wise median of the frozen point snapshot and contains 531 points, giving three real raster events. Crossing observation epochs within one release differs from comparing two revisions: E2025 and E2030 are published projections, not observed history.

Controlled matrix. The controlled matrix uses a 256×256 grid, 8192 points with one point per distinct cell, three regions, 16 events, and 256 changed cells per event. The affected set of each event comprises a shared core of $h \in \{0, 128, 256\}$ cells plus a tail unique to that event; values alternate between two exactly representable integers, so every designated cell really changes. Query gaps are $L \in \{1, 4, 16\}$ events. Events are labelled `synthetic-controlled`.

Point snapshots are identified by `snapshot:<sha256>`; their recorded Overpass base timestamps are retained with the source records.

2) Compared Methods: `BFull-Fast` re-evaluates every point per request from the current state using direct grid lookup, with no inverse index and no cross-query result cache. `B3-Eager` maintains a cell-to-point inverse index and reevaluates affected points at each update. `B3-Lazy` uses the same dependency granularity but accumulates dirty identifiers and evaluates them before query output. `B3-Lazy-demand-init` is an ablation that builds only dependencies at construction. `B2-byte` is a tile cache whose admission is constrained by measured retained state bytes rather than by a guessed tile count.

3) Metrics, Hardware, and Parameters: Correctness is assessed by row-level agreement with a native-array reference and by evidence binding; timed comparisons use the implemented engines. We measure semantic evaluation counts, the paired Lazy/Eager service-time ratio, lifecycle phase costs, per-request amortised cost, retained bytes and budget violations. The historical G0 record reports Windows 10, a 12-logical-CPU Intel64 processor, 17,007,095,808 bytes of memory, Python 3.13.14, NumPy 2.2.6, Shapely 2.1.2, Rasterio 1.4.3 and pyproj 3.7.2. Session reports lack immutable per-run host identifiers, so G0 cannot independently establish the host of every timing. Full performance re-execution on the current host is `NOT_RUN`; the archived protocols, source digests and session results support regeneration. The E1 report records a freeze digest beginning `6075f6c6`, whereas the archived freeze manifest begins `b116834c`; the E1-matching manifest has not been located.

The controlled matrix varies the shared core $h \in \{0, 128, 256\}$ and query gap $L \in \{1, 4, 16\}$. The byte-budget experiment tests $0.5M_0$, M_0 and $2M_0$, where $M_0 = 6.27$ MB for the calibration fixture. The service comparison uses 30 blocks for each dataset, route and schedule; the three engines together yield 720 analysed sessions. Snapshot reuse uses the observed $H = 1, 10, 100$ cases where available.

4) Validity Gates:

a) Correctness gates: Every row binds h_R to the source archive hash and h_P to the point-file hash; a source-only change must alter the suffix while preserving the prefix. Expected rows are rebuilt independently from the native array, validity mask and source transform, using top-down row mapping and exact rational or bit-view encoding rather than the candidate encoder.

b) Performance gates: A session enters a distribution only after all queries and raw validation pass. The freeze covers imported project modules, runtime reads, the protocol, matrix, oracle and registered inputs; one-byte edits to covered files must be rejected. Mutually exclusive phase measurements must close within 1%. Failed sessions are counted and excluded from performance distributions.

5) Statistical Procedure: The unit is a complete session; queries inside a shared process are never independent samples. Engine order within a block is balanced by a fixed seed. Ratios are paired within a block, summarised by the median, and given a 95% block-bootstrap interval (10 000 resamples, seed 20261110). The protocol-specified practical threshold calls an improvement stable only if the interval's upper bound is below 0.95. The archived `round5_evaluation/protocols/R5_PROTOCOL.md` states a freeze date of 2026-11-10 (SHA-256 prefix `7b79f944e3cc1307`); this local date is not an independently timestamped preregistration. Thirty blocks provide the reported median and interval estimates.

B. Comparison with Baselines

1) Complete Service Cost: Across the 720 analysed sessions, no configuration achieves the protocol-specified 5% service improvement (Table I). All eight cells contain 90 passing sessions. The implementation permits at most one retry of an individual session after `TIMING_INVALID`, `CRASH`, or `RESOURCE_LIMIT`, but not after correctness failures; both attempts are retained. One full-evaluation baseline session was retried after `TIMING_INVALID` and passed. The protocol describes rerunning the whole pair, whereas the implementation retried only the failed session; the Lazy/Eager pair was unaffected by this baseline retry. Two registered cells approach the threshold (0.931 and 0.933) but their intervals still include 0.95–0.997, and under W3, where every state is queried, the ratios are 0.975–1.015, consistent with the identity's boundary prediction. Shenzhen also includes insertions and deletions, which are outside the identity's fixed-point premise.

The phase composition explains the result. In the raw route, source verification takes 18.0–18.6% of the lifecycle, decoding 40.6–43.8%, encoding 11.9–20.1%, delta discovery 6.6–12.7% and point preparation 7.2–12.3%; the replay phase that contains all updates and queries takes 0.7% at Shenzhen and 4.2% at São Paulo. Preparation therefore accounts for 93–98% of the lifecycle, and a maintenance change that halves object evaluations cannot move the lifecycle ratio by 5% on this workload.

TABLE I
COMPLETE SERVICE COST, LAZY OVER EAGER, BY DATASET AND ROUTE. 30 PAIRED BLOCKS PER CELL; EVERY SESSION PASSED ALL GATES.

Data	Route	Schedule	valid	median	95% CI
sao	raw	W2_final_only	90/90	1.004	[0.934, 1.046]
sao	raw	W3_each_state	90/90	1.015	[0.982, 1.049]
sao	registered	W2_final_only	90/90	0.931	[0.887, 0.972]
sao	registered	W3_each_state	90/90	1.014	[0.960, 1.053]
shenzhen	raw	W2_final_only	90/90	1.004	[0.949, 1.060]
shenzhen	raw	W3_each_state	90/90	0.975	[0.925, 1.065]
shenzhen	registered	W2_final_only	90/90	0.933	[0.884, 0.997]
shenzhen	registered	W3_each_state	90/90	1.005	[0.939, 1.051]

TABLE II
BYTE-CAPPED TILE CACHE WITH A HARD RETAINED-STATE BUDGET ($M_0 = 6.27\,\mathrm{MB}$). QUERIES PER BUDGET: SIX; ALL CORRECT.

Budget	bytes	p50 (s)	p95 (s)	violations	queries
0.5M0	3136439	0.5388	0.6022	0	6
1.0M0	6272879	0.3927	0.4011	0	6
2.0M0	12545758	0.3810	0.4029	0	6
no budget	–	0.0210	0.0267	–	6

At $H = 1$ the registered route costs 0.74–1.13 times the raw route across cells, because registration performs the same decoding and encoding that the raw route performs. The raw and registered groups were measured in separate passes, so the comparison is descriptive and may reflect cache warm-up. Registration moves preparation to an upfront step.

2) Byte-Budgeted Tile Cache: A tile cache capped by a byte budget stays within every tested memory limit but pays a substantial latency penalty (Table II). The evaluation uses a 100-block persistent trace, and the calibration fixture is a controlled layout with the same size and grid but a different cell assignment; $M_0 = 6.27\,\mathrm{MB}$. At every budget the cache records zero violations, with a p50 query latency of 0.539 s at $0.5M_0$, 0.393 s at M_0 and 0.381 s at $2M_0$; all 18 queries checked against the native reference pass. The six queries per budget belong to one session. For sorted latencies $x_0, \ldots, x_5$, p95 is $x_{\lfloor 0.95(6-1) \rfloor} = x_4$ (the fifth observation). The reported p50 uses x_2 for the tile cache but x_3 for the Lazy reference; their ratio therefore compares different median conventions. The uncapped lazy index answers the same queries with a p50 of 0.021 s. The cache is therefore 18–26$\times$ slower per query in this workload, because a changed cell invalidates a whole tile and the tile is re-evaluated for every point in it, whereas lazy maintenance re-evaluates only the 256 changed objects per event. The byte budget thus controls memory without changing the granularity disadvantage of tile invalidation.

C. Ablation and Mechanism Tests

1) Effect of Deferred Maintenance: Across the controlled matrix, every measured counter equals the identity with zero residual in all 90 runs (five layouts per cell, two engines), and every queried state matches the native reference (Table III). The predictions hold: at $h = 0$ deferral saves nothing at any

TABLE III
CONTROLLED OVERLAP MATRIX: PREDICTED AND MEASURED SEMANTIC EVALUATIONS, 8192 POINTS, 16 EVENTS. RESIDUAL IS ZERO IN EVERY CELL.

h	L	C_E	meas.	C_L	meas.	saving
0	1	12288	12288	12288	12288	0.000
0	4	12288	12288	12288	12288	0.000
0	16	12288	12288	12288	12288	0.000
128	1	12288	12288	12288	12288	0.000
128	4	12288	12288	10752	10752	0.125
128	16	12288	12288	10368	10368	0.156
256	1	12288	12288	12288	12288	0.000
256	4	12288	12288	9216	9216	0.250
256	16	12288	12288	8448	8448	0.312

TABLE IV
PAIRED TIMING AT THE THREE PREREGISTERED ENDPOINTS, 30 BLOCKS EACH, LAZY OVER EAGER (COMPLETE CONTROLLED LIFECYCLE).

Endpoint	sessions	median	95% CI
h=0, L=16	60	1.023	[0.977, 1.059]
h=256, L=1	60	0.990	[0.977, 1.032]
h=256, L=16	60	0.836	[0.804, 0.858]

gap; at $L = 1$ nothing is merged at any overlap; the saving grows with both variables to 31.2% at $h = 256, L = 16$.

The service timings reproduce the same boundary end to end (Table IV). At $h = 0, L = 16$ the paired ratio is 1.023 with an interval spanning 1; at $h = 256, L = 1$ it is 0.990, again spanning 1; at $h = 256, L = 16$ it is 0.836 with an interval entirely below 0.95. The last cell is the only configuration in this study whose service-level improvement passes the protocol-specified threshold, and it is exactly the cell that the identity predicts.

2) Demand Initialization: The demand-initialization variant constructs the dependencies needed for updates but postpones the initial materialisation until a query. In the recorded controlled endpoints $(h, L) = (0, 16), (256, 1), (256, 16)$, its measured evaluation counts are 8,192, 12,032 and 8,192, respectively, each matching the expected count with zero residual. The first materialisation evaluates 8,192 points and subsumes the events in its first group. These results establish the evaluation-count behaviour at the recorded endpoints.

3) Evidence-Binding Negative Controls: All eight gates pass. The closure contains 165 files and rejects 7 of 7 one-byte negative controls, including a direct source file, the registered round-2 window, the oracle, the protocol, both manifests and the parameter matrix. Eight of eight protocol-specified value classes behave as specified: 0.5 is exactly encodable, 0.1 and the São Paulo value are rejected with a denominator message, ± 0 are distinguished, the sentinel never reaches the encoder, and NaN and infinity are rejected; the hexadecimal encoding agrees with the independent bit view on every finite case. The provenance fixture confirms the intended asymmetry: identical values behind different archive bytes produce an identical semantic prefix and a different suffix, and an implementation that emits the stale hash is caught. The native reference

TABLE V
REGISTERED-ROUTE AMORTIZATION; COST IS SECONDS PER REQUEST. RATIOS USE $H = 1$ AS BASELINE; BRACKETS GIVE EXPLORATORY POST-HOC 95% UNPAIRED SESSION-BOOTSTRAP INTERVALS. SP: SÃO PAULO; SZ: SHENZHEN.

Data	Engine	H	n	Cost	Ratio	95% interval
SP	Eager	1	30	3.419	–	–
SP	Eager	10	7	0.560	6.10	[5.62, 6.39]
SP	Eager	100	7	0.275	12.43	[11.59, 12.87]
SP	Lazy	1	30	3.163	–	–
SP	Lazy	10	7	0.471	6.71	[6.36, 7.03]
SP	Lazy	100	7	0.179	17.70	[17.14, 18.41]
SP	Full	1	30	3.051	–	–
SZ	Eager	1	30	1.805	–	–
SZ	Eager	10	7	0.254	7.10	[6.22, 8.36]
SZ	Eager	100	7	0.059	30.68	[27.16, 35.53]
SZ	Lazy	1	30	1.609	–	–
SZ	Lazy	10	7	0.235	6.84	[6.28, 7.54]
SZ	Lazy	100	7	0.055	29.50	[27.19, 32.95]
SZ	Full	1	30	1.673	–	–

independently reproduces the specified row counts exactly (174, 217 and 243).

The stale-evidence fixture tests whether evidence binding detects an invalid source suffix.

D. Analysis Experiments

1) Preparation Cost and Snapshot Reuse: Reusing a registered snapshot reduces per-request cost by 12–31× at $H = 100$ in the original analysis (Table V; $n = 7$ sessions per cell versus $n = 30$ at $H = 1$). The postprocessor excluded blocks 0–2 as smoke runs at every H, although the $H = 10$ and $H = 100$ loops each recorded ten passing sessions without a separate smoke designation. Including all ten gives $H = 100$ ratios of 12.34–30.22×. Intervals are exploratory post-hoc independent session bootstraps (10 000 draws, seed 20261112): $H = 1$ and $H > 1$ were measured in separate passes. The cost is the sum of separate registration and online medians, divided by H. These results measure preparation amortization; the controlled paired experiment in Table IV measures deferred maintenance.

The controlled overlap and query-gap matrix in Section V-C also serves as a parameter analysis: the maximum observed reduction in evaluation count is 31.2%. Paired timings quantify the corresponding service behaviour.

2) Change Discovery and Scale: Runtime grows with cells, points and regions for different reasons (Table VI). Holding points and regions fixed, a $16\times$ increase in cells raises the total from 0.81 s to 3.77 s, while the evaluation count stays near 10 100 because only the affected points are evaluated; the cost is in encoding and in the full-snapshot comparison. Holding cells and regions fixed, a $100\times$ increase in points raises the total from 0.19 s to 6.85 s and the evaluation count from 1 011 to 102 515. Holding cells and points fixed, a $100\times$ increase in regions raises the total from 0.74 s to 22.83 s with the same evaluation count, because region membership is tested per point per region. Delta discovery from complete snapshots costs 0.9 ms at this scale against 0.4 ms for parsing and validating an equivalent event log, so the log is cheaper only

TABLE VI
ONE-AXIS SCALE TESTS (SINGLE RUN PER CONFIGURATION, USED FOR COST EXPLANATION). EVERY ROW MATCHED THE NATIVE REFERENCE.

Axis	cells	points	regions	eval	total (s)	ok
cells	65536	10000	3	10113	0.81	True
cells	262144	10000	3	10101	1.33	True
cells	1048576	10000	3	10101	3.77	True
points	65536	1000	3	1011	0.19	True
points	65536	10000	3	10113	0.75	True
points	65536	100000	3	102515	6.85	True
regions	65536	10000	3	10113	0.74	True
regions	65536	10000	30	10113	2.92	True
regions	65536	10000	300	10113	22.83	True

TABLE VII
REAL V2–V3 MOVEMENT: RASTER × POINT VERSION MATRIX, EVERY ENGINE COMPARED ROW BY ROW AGAINST THE NATIVE REFERENCE.

raster	point	engine	rows	agrees
R2022A	2	BFull-Fast	2	yes
R2022A	2	B3-Eager	2	yes
R2022A	2	B3-Lazy	2	yes
R2022A	2	B2-byte	2	yes
R2022A	3	BFull-Fast	2	yes
R2022A	3	B3-Eager	2	yes
R2022A	3	B3-Lazy	2	yes
R2022A	3	B2-byte	2	yes
R2023A	2	BFull-Fast	2	yes
R2023A	2	B3-Eager	2	yes
R2023A	2	B3-Lazy	2	yes
R2023A	2	B2-byte	2	yes
R2023A	3	BFull-Fast	2	yes
R2023A	3	B3-Eager	2	yes
R2023A	3	B3-Lazy	2	yes
R2023A	3	B2-byte	2	yes

when a producer already supplies it; offline log construction is measured separately.

3) Real Movement and Failure Checks: The selected OSM node moved from $(10902565.9, 2728589.1)$ to $(10902607.2, 2728560.0)$ on the Mollweide grid, one cell to the east. The minimal native window covering both cells plus two cells of margin is 5×6 cells, and both positions are covered by the real R2022A and R2023A tiles. All 16 combinations of raster version, point version and engine reproduce the native reference row by row, including two postselected regions constructed to separate the old and new positions: the emitted cell identifier changes from 2:2 to 2:3 while the semantic prefix otherwise stays fixed, and the source suffix follows the requested versions. In one release the old and new sampled values are equal, while the cell identifier still records the movement.

On serial event sequences, the event guard accepts an ordered $v1 \to v2 \to v3$ chain and acknowledges an identical committed event idempotently. It rejects an event whose sequence is not contiguous, an event whose base is not current, an update after deletion, and the same event identifier with a different payload; in every rejection the snapshot and digest are unchanged, and intermediate states remain observable.

VI. DISCUSSION AND REPRODUCIBILITY

The evaluation covers a Shenzhen same-epoch revision pair and a São Paulo sequence across observation epochs. The controlled matrix isolates the effects of overlap and query spacing. The evaluated real windows contain no NoData transitions. In a post-hoc synthetic fixture executed on the current host, valid-to-NoData and NoData-to-valid changes were exercised across four engines and three states, yielding 12 of 12 row-set checks against the reference. The event-guard tests cover ordered events and rejection of invalid transitions. The R2022A tile is retained as a historical archive for the revision comparison.

The experiments compare the four implemented engines and retain the source data, code and reference outputs needed to examine their behaviour.

The reproduction artifact is available at https://github.com/dominodu0828/GeoScopeJoin, with the versioned package in the repository's Releases. It contains the original source archives and point snapshots, source retrieval records, code and input hashes, frozen protocols, per-session results, negative-control outputs, and table generators. GHSL and OpenStreetMap materials retain their source attribution and licenses. The archived R2022A tile is included for the historical revision comparison. A machine-readable ledger links each reported claim to its run and artifact. Historical measurements are preserved separately from publication validation; users regenerate the execution closure for their local environment before rerunning the experiments.

VII. CONCLUSION

Exact spatial reuse depends on a contract that preserves object multiplicity, boundaries, missing-state distinctions and source identity. This paper makes that contract executable, binds it to registered bytes, and determines when maintenance can pay. For a fixed point set, eager and deferred maintenance differ by exactly the coalescing of affected sets between queries: the benefit in evaluation counts is zero without overlap or without deferral and grows with both. The experiments confirm the identity to zero residual and validate its predictions at the service level. On the real one-shot workloads, preparation consumes 93–98% of the lifecycle and maintenance and querying account for 0.7–4.2%, explaining the absence of a 5% end-to-end improvement. Reusing registered snapshots over 100 requests reduces per-request cost by 12–31×, while controlled paired timings isolate the benefit of deferral itself. The experiments also quantify the trade-off between byte-budgeted tile caching and object-level lazy maintenance. The result is an executable contract for correctness, a predictive identity, and a lifecycle account that identifies where exact spatial reuse provides practical value.

ACKNOWLEDGMENT OF AI-GENERATED CONTENT

In Round 5, we used OpenAI Codex, ZCode, Kimi, and DeepSeek. We used these tools to write and audit experiment code, run measurements, analyze results, generate reports and tables, validate the reproduction package, and draft and revise

the manuscript. The authors take responsibility for the code, results, references, and manuscript.

REFERENCES

[1] A. Gupta, I. S. Mumick, and V. S. Subrahmanian, "Maintaining views incrementally," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1993, pp. 157–166.

[2] S. Singla, A. Eldawy, T. Diao, A. Mukhopadhyay, and E. Scudiero, "The Raptor join operator for processing big raster + vector data," in *Proceedings of the 29th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2021.

[3] Z. Shang, S. Singla, A. Eldawy, and E. Scudiero, "RDPro: Distributed processing of big raster data: [scalable data science]," *Proceedings of the VLDB Endowment*, vol. 18, no. 3, pp. 613–622, 2024.

[4] X. Niu, B. Glavic, Z. Liu, P. Li, D. Gawlick, V. Krishnaswamy, Z. H. Liu, and D. Porobic, "Provenance-based data skipping," *Proceedings of the VLDB Endowment*, vol. 15, no. 3, pp. 451–464, 2021.

[5] P. Li, B. Glavic, D. Gawlick, V. Krishnaswamy, Z. H. Liu, D. Porobic, and X. Niu, "In-memory incremental maintenance of provenance sketches [extended version]," arXiv:2505.20683, 2025. [Online]. Available: https://arxiv.org/abs/2505.20683